

CSP584 - EWA Project

ELECTRO STORE - PHASE 2

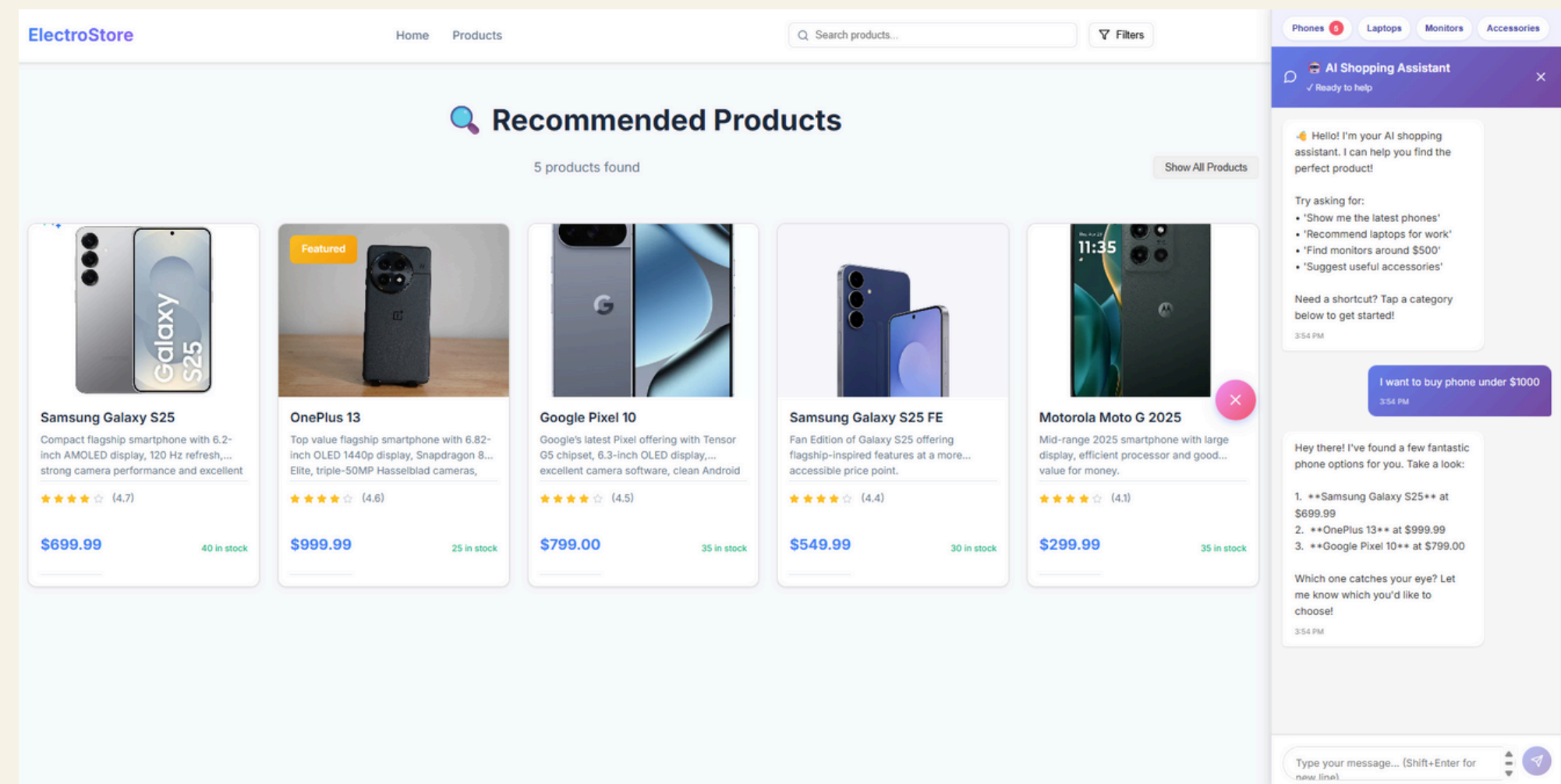
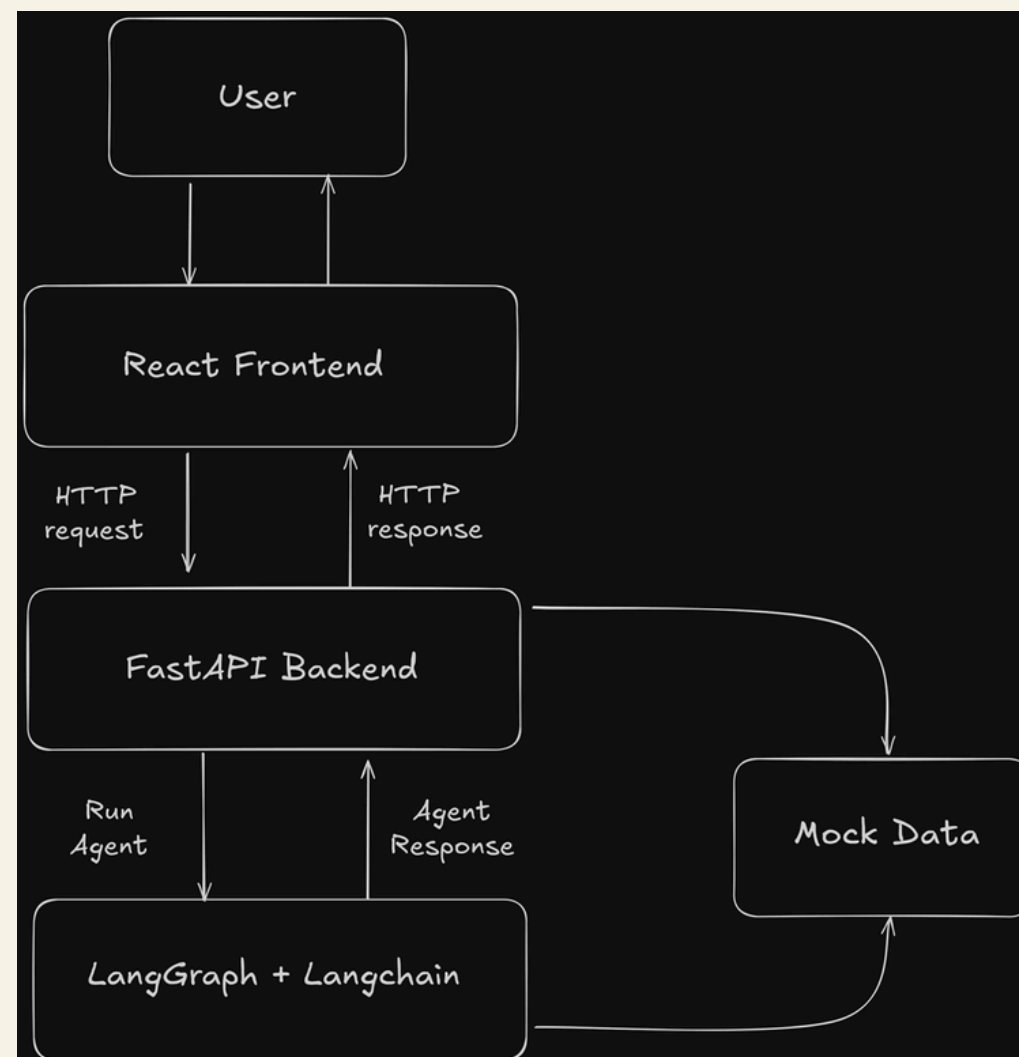
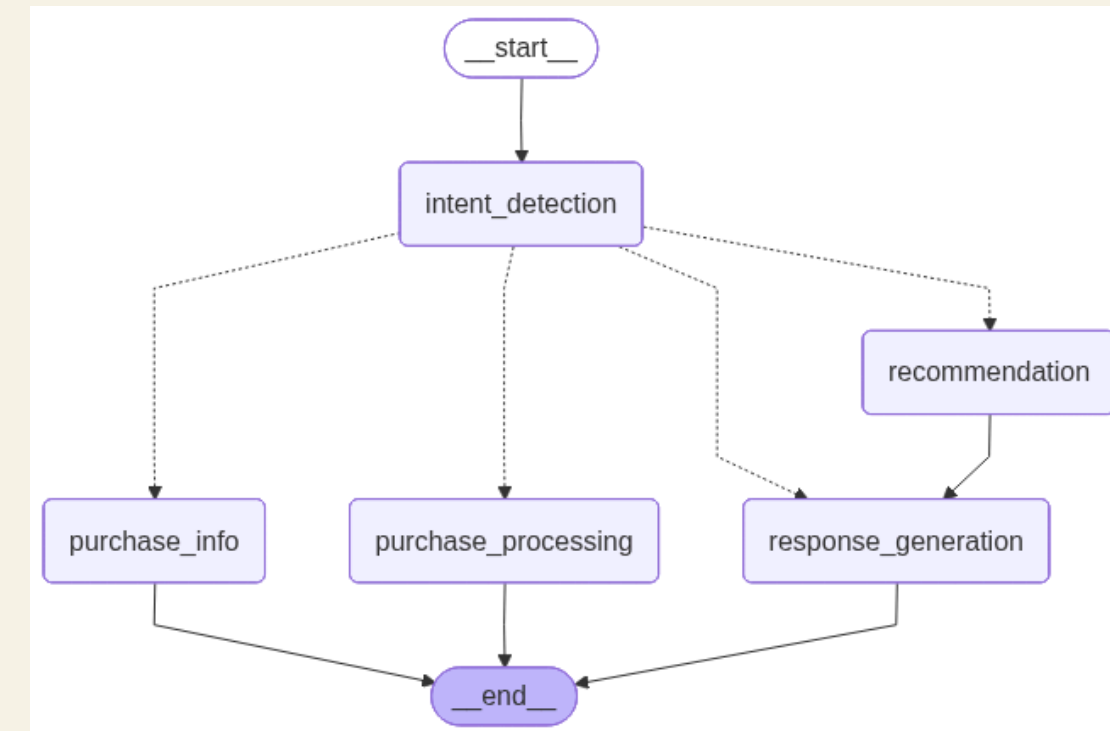
Seamless Shopping Experience with
Intelligent Cart & AI Chatbot

Team #7:

A20577685 - Nijgururaj Ashtagi

Phase 1 Recap

- Successfully built and deployed a modular pipeline integrating LLM-driven product recommendations.
- Demonstrated real-time filtering and response handling using structured LLM outputs.
- Improved code modularity and error resilience with clear payload-response handling.



Phase 2 Scope

Two more Agents

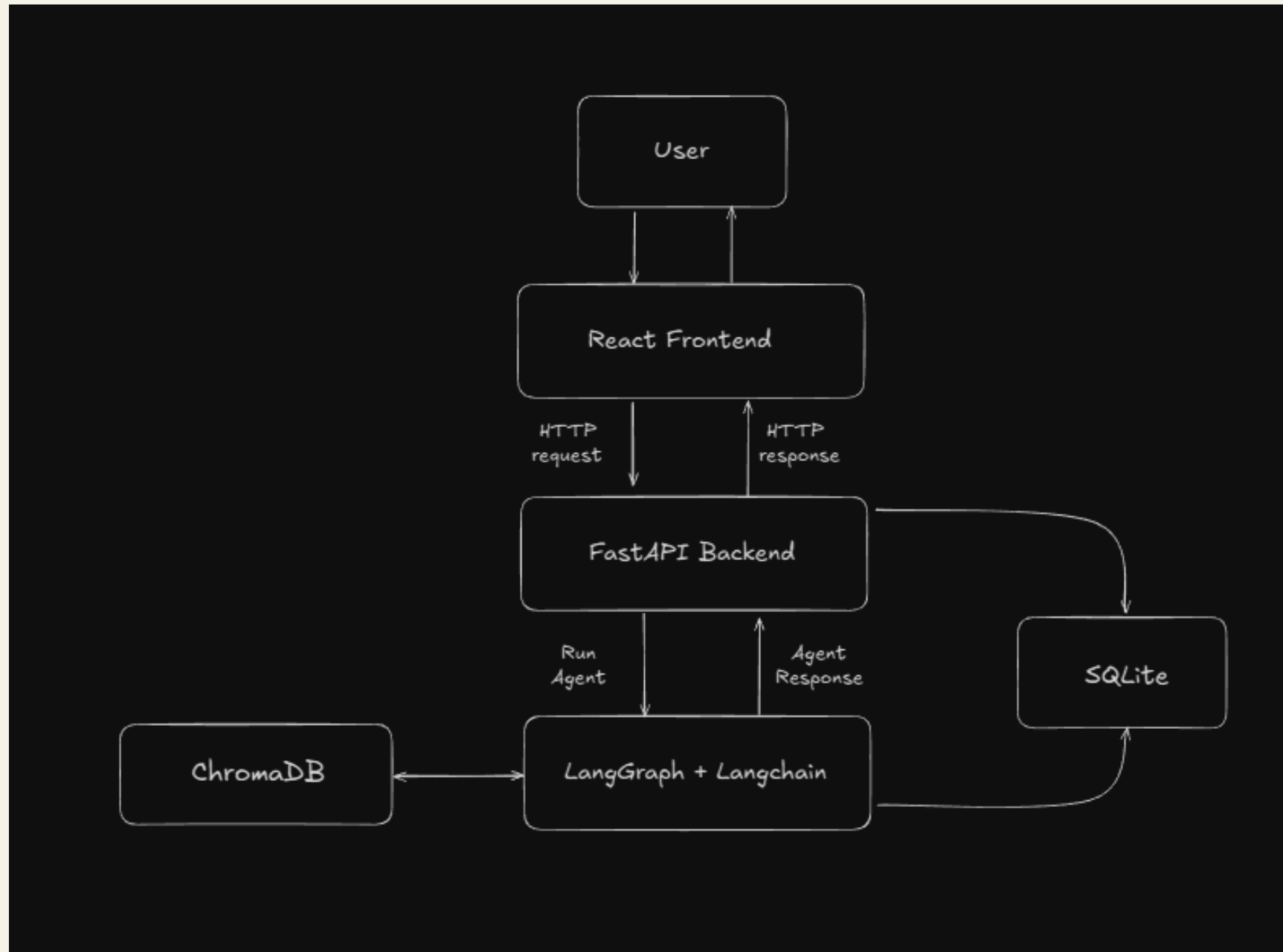
- **Order and Shipping Status Agent**

This agent helps users track and resolve issues with a delivered order. When a customer asks about the status of their purchase, the agent retrieves the order information from the backend and provides the current shipping or delivery status. If the customer reports a problem and uploads an image of a defective product or a damaged shipping box, the agent analyzes the image along with the order details to decide whether to issue a replacement, refund, or escalate the case to a human agent.

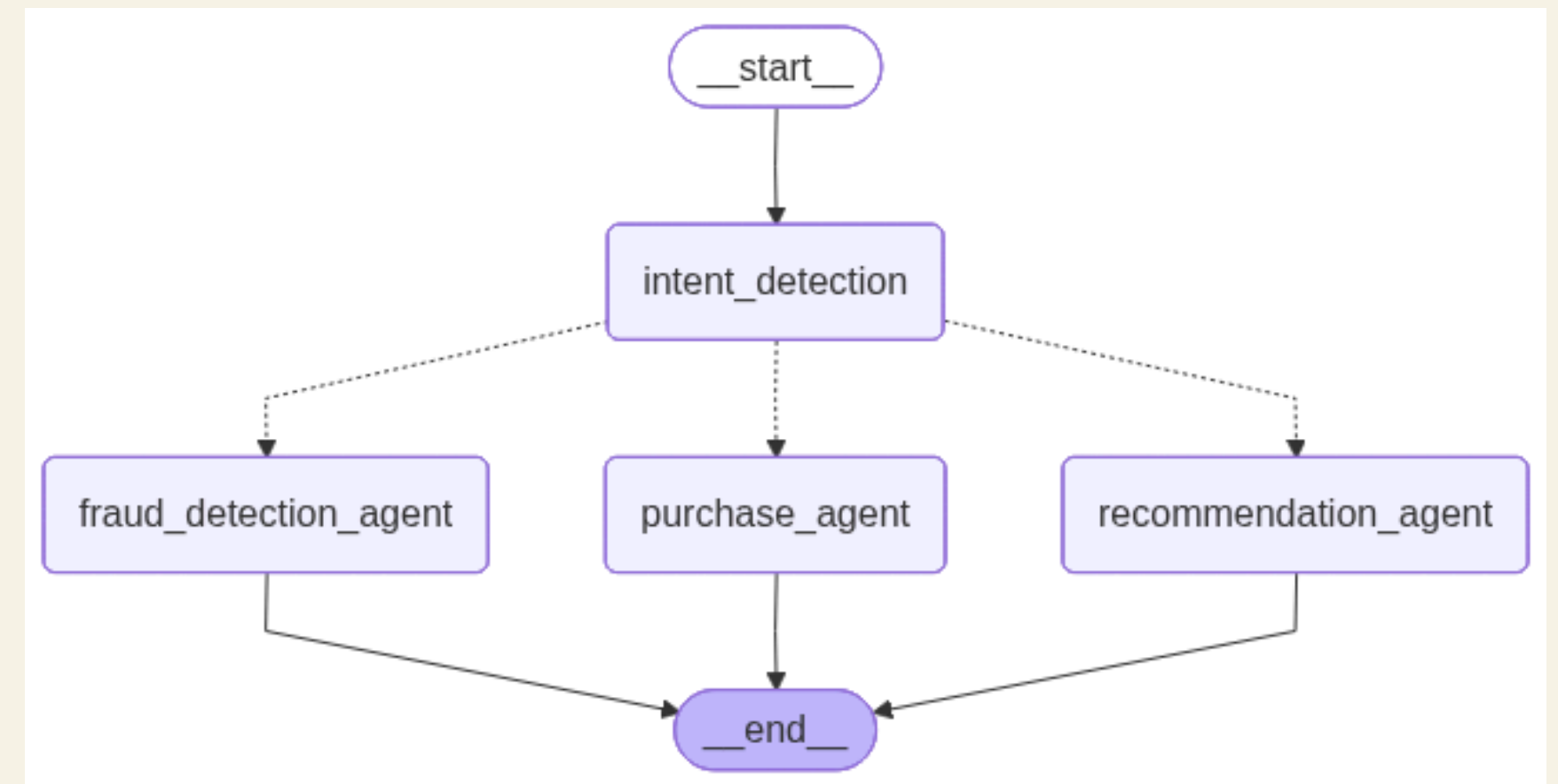
- **Fraudulent Transaction Agent**

This agent supports customers who report suspicious or unauthorized purchases. The user submits an image of the transaction information. OCR technology extracts key details such as name, amount, and transaction ID. The agent evaluates this data against stored order history to determine if the transaction should be refunded, declined, or escalated to a human agent for further investigation.

Phase 2 Architecture



High Level Architecture



Agent Architecture

Tech Stack - Backend

* Frontend tech stack is same as before

Layer	Technology	Description
Core Framework	FastAPI	Modern Python web framework for backend APIs
	Pydantic	Data validation and typing
	Uvicorn	ASGI server running the FastAPI app
AI and ML Layer	LangGraph	Agent workflow and decision routing
	LangChain	Orchestration of tools and model interactions
	OpenAI (gpt-4o-mini)	Language model for dialog and reasoning
	OCR Module (OpenAI - Vision LLM) (gpt-4o)	Extracts text from uploaded images for fraud checks
Database and Storage	SQLite	Stores users, orders, carts, products, complaints, and sessions
	Vector Store (ChromaDB)	Saves product embeddings and supports faster retrieval for recommendations
Authentication	PyJWT	JWT generation and verification
	HTTPBearer	Request level token authentication

Screenshots for Fraud Detection

Filters

PhonesLaptopsMonitors

AI Shopping Assistant

Ready to help

around \$500 • 'Suggest useful accessories'

Need a shortcut? Tap a category below to get started!

2:59 PM

Featured

Dell 27-inch 4K Monitor (S2725QS)

27-inch 4K UHD display, 120 Hz refresh rate, 99% sRGB colour accuracy — ideal for productivity and light creative work.

★★★★☆ (4.6)

\$239.99

15 in stock

Received a broken Phone in the box

3:00 PM

I've analyzed the image of your defective product. Based on the severe defect (Severely cracked back glass), I've determined that a refund is the appropriate action.

Reason: The image shows a phone with a severely cracked back, indicating a severe defect likely due to shipping.

Your refund will be processed within 5-7 business days. Is there anything else I can help you with?

Refund Approved

Reason: The image shows a phone with a severely cracked back, indicating a severe defect likely due to shipping.

Case ID: 31574F67

3:00 PM

Type your message... (Shift+Enter for new line)

Filters

PhonesLaptopsMonitors

AI Shopping Assistant

Ready to help

Received a damaged shipping box

3:03 PM

Featured

Dell 27-inch 4K Monitor (S2725QS)

27-inch 4K UHD display, 120 Hz refresh rate, 99% sRGB colour accuracy — ideal for productivity and light creative work.

★★★★☆ (4.6)

\$239.99

15 in stock

I've analyzed the image of your damaged shipping box. Based on the moderate box damage and the product condition (possibly_damaged), I've determined that a replacement is the appropriate action.

Reason: The box is moderately damaged, which could potentially affect the product inside.

A replacement product will be shipped to you within 2-3 business days. You'll receive a tracking number via email. Please return the damaged item using the prepaid return label that will be included. Is there anything else I can help you with?

Replacement Approved

Reason: The box is moderately damaged, which could potentially affect the product inside.

Case ID: AE386ABD

3:03 PM

Type your message... (Shift+Enter for new line)

Filters

PhonesLaptopsMonitors

AI Shopping Assistant

Ready to help

I never did the transactions still my card was debited and got this in the mail

3:05 PM

Featured

Dell 27-inch 4K Monitor (S2725QS)

27-inch 4K UHD display, 120 Hz refresh rate, 99% sRGB colour accuracy — ideal for productivity and light creative work.

★★★★☆ (4.6)

\$239.99

15 in stock

I've reviewed your fraudulent transaction report. This case requires additional review by our fraud investigation team.

Transaction Amount: \$1,409.42

Reason: The transaction receipt shows a high-value purchase, and the user claims it was unauthorized.

A fraud specialist will contact you within 24 hours to investigate this matter further. Your case reference number is: B9C6018F. In the meantime, we recommend monitoring your account and contacting your bank if needed. Is there anything else I can help you with?

Escalated to Support

Reason: The transaction receipt shows a high-value purchase, and the user claims it was unauthorized.

Case ID: B9C6018F

3:05 PM

Type your message... (Shift+Enter for new line)

Fraud Detection Tools

```
def analyze_product_defect_image(image_url: str, user_message: str) -> Dict[str, Any]:
    """
    Analyzes an image of a product defect and determines the appropriate action:
    Refund, Replace, or Escalate to Human-Agent.
    """

    if not vision_llm:
        return {"decision": "escalate", "reason": "Vision model not available"}

    prompt = get_product_defect_analysis_prompt()

    try:
        # Handle base64 or URL images
        if image_url.startswith("data:image"):
            # Base64 encoded image
            image_content = image_url
        elif image_url.startswith("http"):
            # URL - we'll pass it as is
            image_content = image_url
        else:
            # Assume base64 without data URI prefix
            image_content = f"data:image/jpeg;base64,{image_url}"

        message = HumanMessage(
            content=[
                {"type": "text", "text": prompt.format(user_message=user_message)},
                {"type": "image_url", "image_url": {"url": image_content}}
            ]
        )

        parser = JsonOutputParser()
        chain = vision_llm | parser
        result = chain.invoke([message])
        return result
    except Exception as e:
        print(f"Error analyzing product defect image: {e}")
        return {"decision": "escalate", "reason": f"Error analyzing image: {str(e)}"}
```

```
def analyze_damaged_shipping_box_image(image_url: str, user_message: str) -> Dict[str, Any]:
    """
    Analyzes an image of a damaged shipping box and determines the appropriate action:
    Refund, Replace, or Escalate to Human-Agent.
    """

    if not vision_llm:
        return {"decision": "escalate", "reason": "Vision model not available"}

    prompt = get_damaged_box_analysis_prompt()

    try:
        if image_url.startswith("data:image"):
            image_content = image_url
        elif image_url.startswith("http"):
            image_content = image_url
        else:
            image_content = f"data:image/jpeg;base64,{image_url}"

        message = HumanMessage(
            content=[
                {"type": "text", "text": prompt.format(user_message=user_message)},
                {"type": "image_url", "image_url": {"url": image_content}}
            ]
        )

        parser = JsonOutputParser()
        chain = vision_llm | parser
        result = chain.invoke([message])
        return result
    except Exception as e:
        print(f"Error analyzing damaged shipping box image: {e}")
        return {"decision": "escalate", "reason": f"Error analyzing image: {str(e)}"}
```

```
def analyze_fraudulent_transaction_ocr(image_url: str, user_message: str) -> Dict[str, Any]:
    """
    Analyzes an OCR image of a credit card statement and determines the appropriate action:
    Refund, Decline, or Escalate to Human-Agent.
    """

    if not vision_llm:
        return {"decision": "escalate", "reason": "Vision model not available"}

    prompt = get_fraud_analysis_prompt()

    try:
        if image_url.startswith("data:image"):
            image_content = image_url
        elif image_url.startswith("http"):
            image_content = image_url
        else:
            image_content = f"data:image/jpeg;base64,{image_url}"

        message = HumanMessage(
            content=[
                {"type": "text", "text": prompt.format(user_message=user_message)},
                {"type": "image_url", "image_url": {"url": image_content}}
            ]
        )

        parser = JsonOutputParser()
        chain = vision_llm | parser
        result = chain.invoke([message])
        return result
    except Exception as e:
        print(f"Error analyzing fraudulent transaction OCR: {e}")
        return {"decision": "escalate", "reason": f"Error analyzing image: {str(e)}"}
```

Analysis and Evaluations

Strategy	Description	Impact
Image Based Decision Pipeline	Image analysis for damaged items and boxes	More accurate automated resolution decisions
OCR Accuracy Improvements	OCR for transaction screenshots with basic correction	Improved fraud validation and fewer false cases
Retrieval Driven Reasoning	Vector store retrieval for product matching	Faster and more precise decision support
Multi Agent Coordination	Clear coordination among three agents	Stable routing and consistent outcomes
Persistent Data Layer	Shift to SQLite for persistent records	Reliable data tracking and easier evaluation

Conclusion & Findings

- Phase 2 expanded the system into a full post purchase assistance platform
- New agents successfully handle damaged product and fraud scenarios using uploaded images
- OCR and vector retrieval improved the accuracy of automated decisions
- Persistent SQLite storage enabled reliable tracking of users, orders, complaints, and sessions
- Overall results show strong potential for combining language models with visual analysis in customer service
- Future enhancements include better OCR correction, confidence scoring, and optimized defect detection

References

Backend Frameworks

1. Tiangolo, Sebastián. FastAPI Documentation. FastAPI, 2025, <https://fastapi.tiangolo.com/>.
2. Pydantic Developers. Pydantic Documentation. Pydantic, 2025, <https://docs.pydantic.dev/>.
3. Uvicorn Developers. Uvicorn Documentation. Encode, 2025, <https://www.uvicorn.org/>.
4. SQLite Consortium. SQLite Documentation. SQLite, 2025.
<https://docs.sqlalchemy.org/en/20/dialects/sqlite.html>

AI / LLM Frameworks

1. OpenAI. OpenAI API Documentation. OpenAI, 2025, <https://platform.openai.com/docs>.
2. OpenAI. Vision Capabilities Documentation. OpenAI, 2025,
<https://platform.openai.com/docs/guides/images-vision>
3. LangChain AI. LangGraph Documentation. LangChain, 2025, <https://langchain-ai.github.io/langgraph/>.
4. LangChain AI. LangChain Documentation. LangChain, 2025, <https://python.langchain.com/>.

Vector Storage

1. ChromaDB Team. Chroma Documentation. ChromaDB, 2025.
<https://docs.langchain.com/oss/python/integrations/vectorstores/chroma>

Frontend Frameworks

1. React Community. React Documentation. Meta Platforms, 2025, <https://react.dev/>.
2. Axios Contributors. Axios Documentation. GitHub, 2025, <https://axios-http.com/>.

THANK YOU